

WhatsApp Decryption Flows and Test Cases

Mobile extraction → matching key intake → derived evidence → RAG and reports

Supported: original .crypt; CRYPT5, 7, 8, 9, 10, 11, 12, 14 and 15.

Includes a runnable synthetic CRYPT14 case and missing/wrong-key behavior.

A CRYPT14 file alone cannot yield its key. Unsupported versions remain visible as encrypted evidence.

Contents and diagram key

WhatsApp / Supported formats and executable test case	3
Key intake / what the CRYPT14 test actually supplies	4
Modern CRYPT12 / CRYPT14 / CRYPT15 decryption	5
WhatsApp / Historical CRYPT formats	6
WhatsApp / Derived payloads before artifact parsing	7
Recovered data / artifacts, RAG and report evidence	8
Run the supplied case / Windows or Python environment	9
Failure cases / honest results for every source	10
Verification scope / performance and live acceptance	11

Diagram key

 Process	 Gate / branch	 Retained evidence
 Key / crypto	 Gap / failure	 External workflow

Solid arrows show implemented transitions. Dashed arrows show an explicitly external or conditional path. A gap is evidence of a limitation; it does not establish that the requested data never existed.

WhatsApp / Supported formats and executable test case

The public QA files are synthetic. The CRYPT14 case includes its matching 158-byte key file; the keyless case must remain blocked.

Format	Required material	Validation / result
Original .crypt	Historical public AES format key	AES-192-ECB; complete SQLite structural checks; authenticity unverified
CRYPT5	Already-derived 24-byte / 48-hex key, or exact original Google account	AES-192-CBC; complete SQLite + quick_check; no authentication tag
CRYPT7 / CRYPT8	Matching files/key or 32-byte AES secret	AES-256-CBC; CRYPT8 gzip; complete SQLite checks; authenticity unverified
CRYPT9 / 10 / 11	Matching files/key or raw 32-byte AES secret	GCM tag when present; exact-EOF historical tagless streams remain unverified; failed tags never discarded
CRYPT12 / CRYPT14	Matching files/key; raw / hexadecimal secret accepted	AES-GCM tag and any MD5 footer required; bounded expansion
CRYPT15	Matching encrypted_backup.key or 64-character root secret	HKDF-SHA256 then AES-GCM authentication and footer validation
Other CRYPT extensions	No validated decoder in this release	Retain encrypted bytes; unsupported_crypt_version; never assume another format

Run From the updated project: `python scripts/test-whatsapp-decryption.py --out C:/Aetheris-QA/whatsapp-case` (choose an empty output folder).

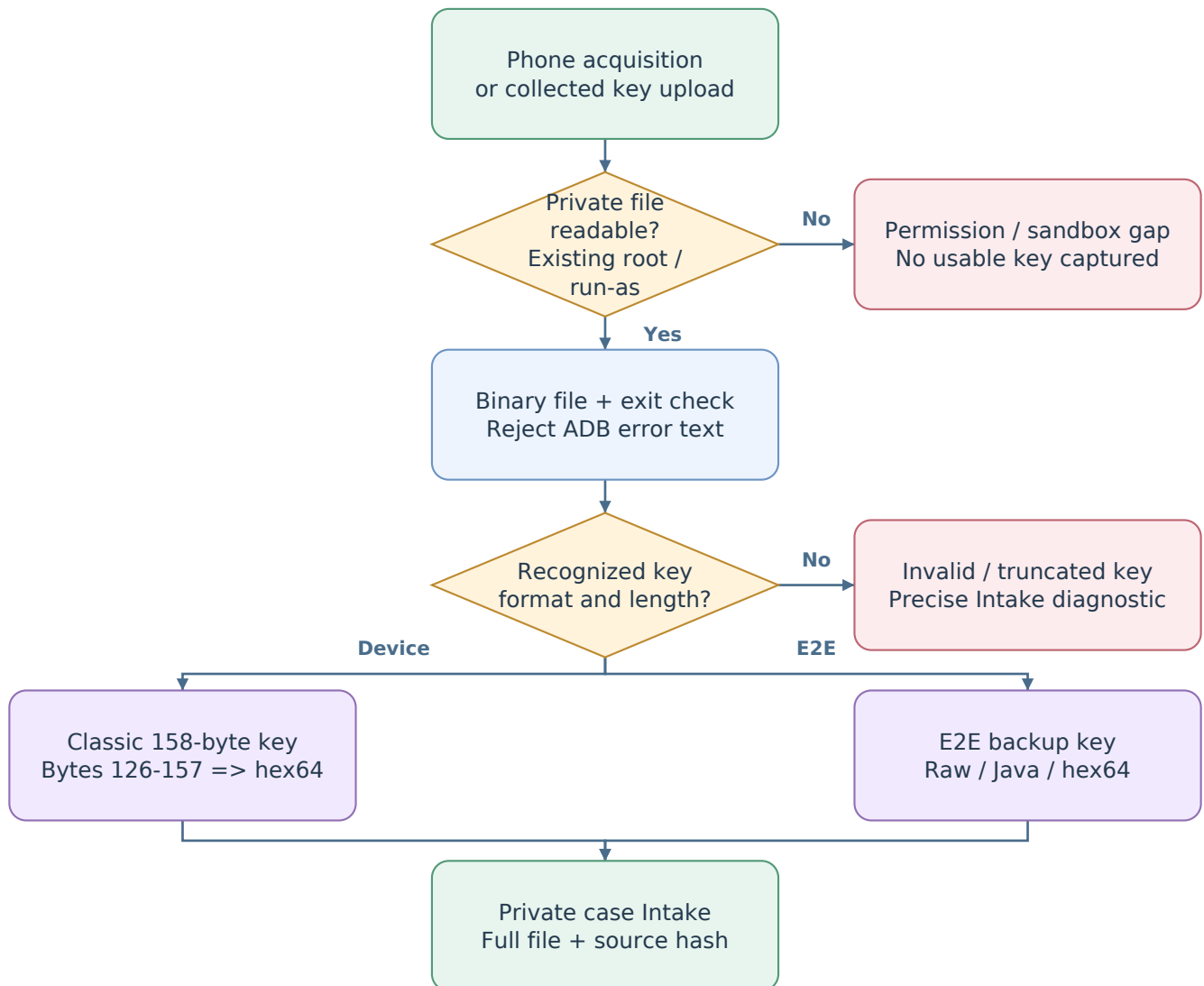
Checks The script checks ten formats, typed payloads, key capture offset, keyless blocking, chats, a surviving deleted row, attachment reference, RAG-ready text and unchanged input hashes.

Live status Synthetic and PostgreSQL-compatible checks passed. No connected Android phone, native Windows device acquisition or GPU session was available for live acceptance.

Implementation: `scripts/whatsapp_qa_fixtures.py`; `scripts/test-whatsapp-decryption.py`; `backend/tests/test_whatsapp_payloads.py`

Key intake / what the CRYPT14 test actually supplies

A CRYPT14 backup does not contain its secret. Acquisition captures the matching private file when Android access permits it, or an examiner supplies that already-collected file.



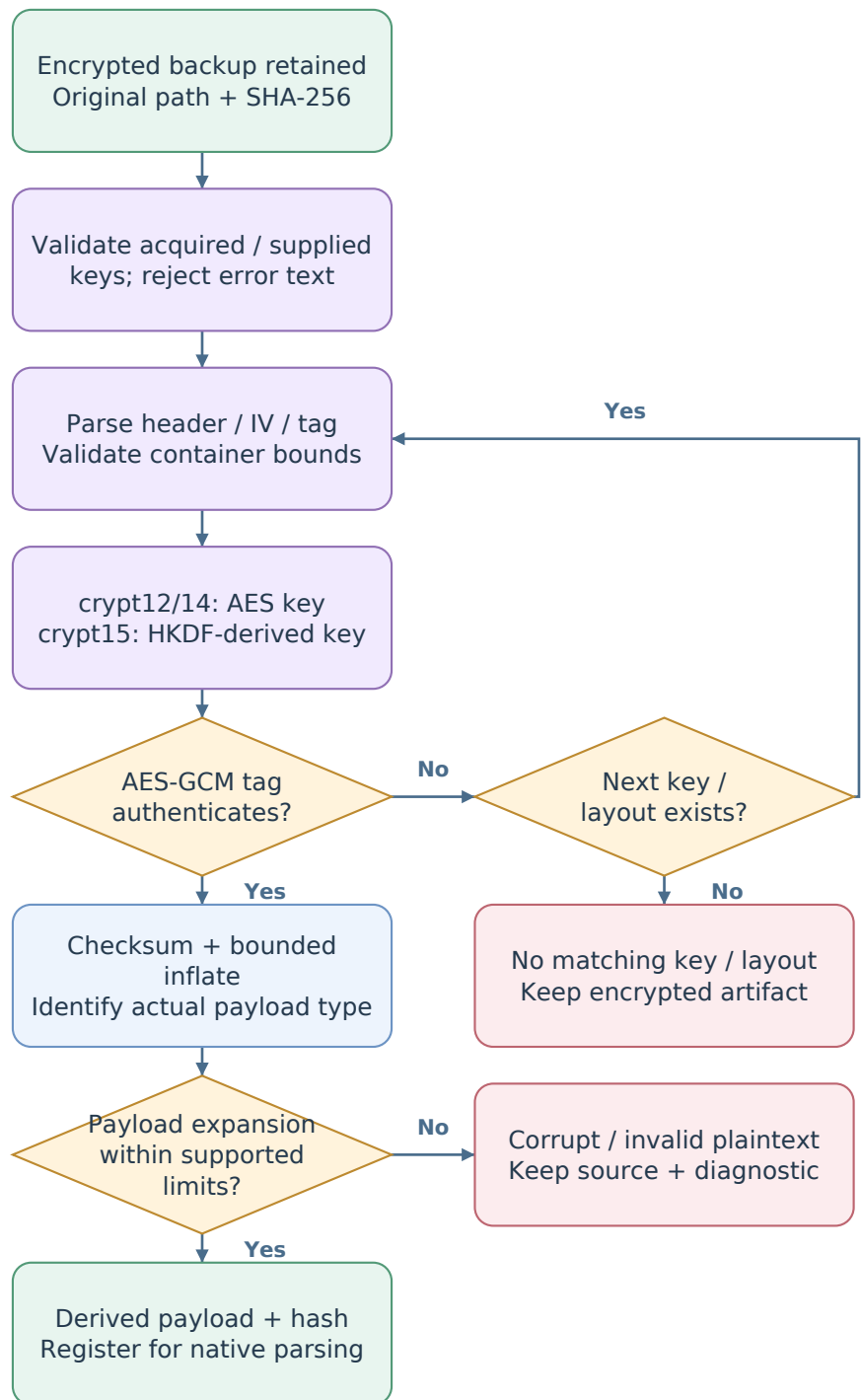
Path The classic source is `/data/data/com.whatsapp/files/key`; user profiles and `com.whatsapp.w4b` variants are also captured when readable. CRYPT15 uses `encrypted_backup.key`.

Exact requirement The 64 hexadecimal characters come from the collected key file, not the CRYPT14 ciphertext. Different installations/backups may need different secrets.

Implementation: `mobile_acquire/privileged_app_pull.py`; `mobile_forensic/key_intake.py`; `routers/report.py`

Modern CRYPT12 / CRYPT14 / CRYPT15 decryption

Modern backups authenticate before payload expansion. Payloads can contain SQLite databases, incremental ZIPs, settings, images or binary data.



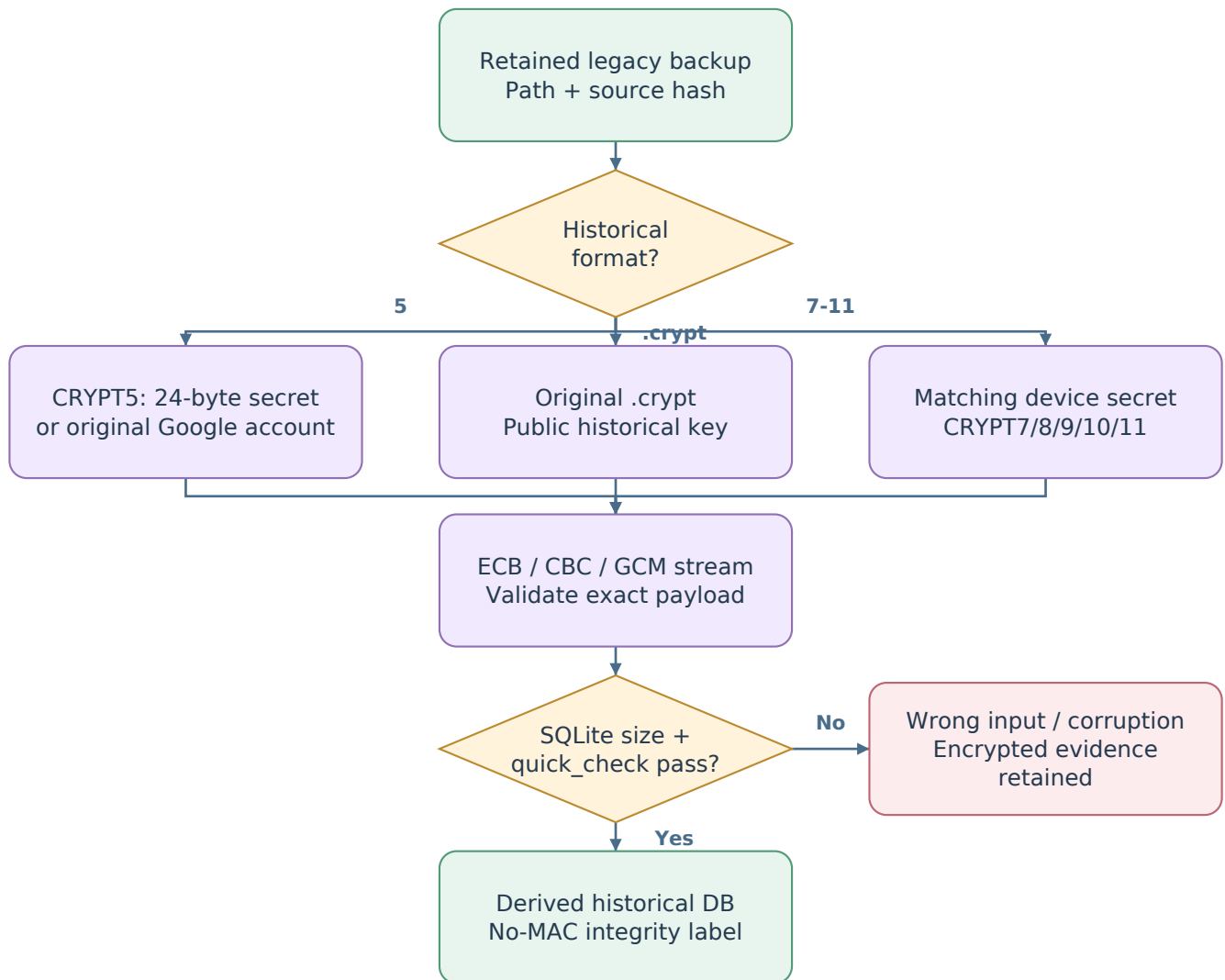
Key formats A 158-byte Android key file supplies its last 32 bytes (offset 126). Raw 32-byte / 64-hex keys and recognized Java byte-array envelopes are validated.

Crypto boundary CRYPT15 uses HKDF-SHA256. Every key candidate is tried per source, including Business and Android profiles. Protobuf IVs avoid repeated historical guesses; source/key-set cache avoids repeat decryption.

Implementation: backend/app/services/mobile_forensic/whatsapp_crypt.py; whatsapp_derivation.py

WhatsApp / Historical CRYPT formats

Historical ECB, CBC and tagless stream backups do not establish cryptographic authenticity. Recovery verifies SQLite structure and records the integrity limit on every parsed chat.



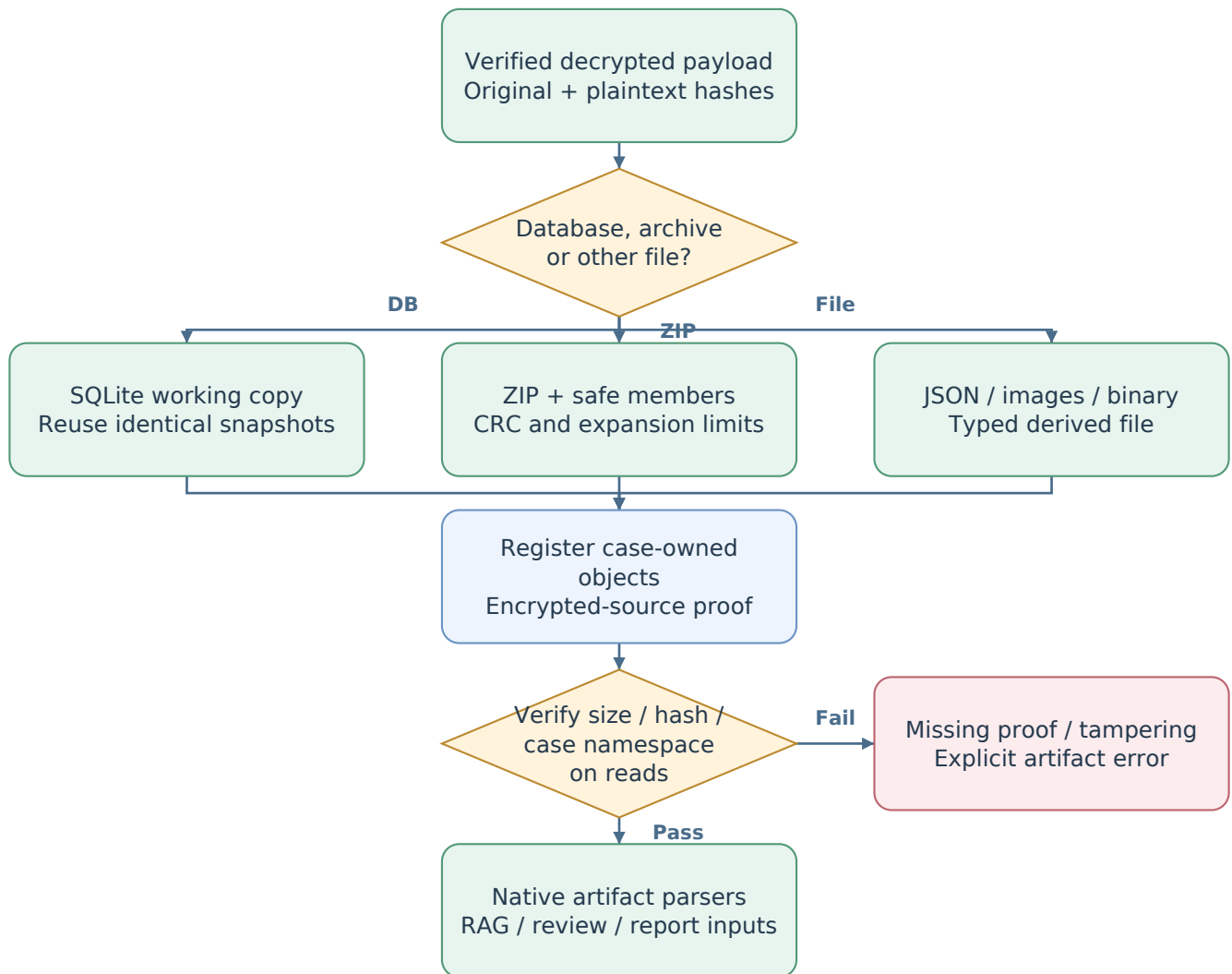
Meaning Structural checks do not prove authenticity. Legacy chat records carry a medium confidence ceiling and the legacy_no_authentication_tag validation note.

Account CRYPT5 account derivation is specific to this old format. It does not extract a CRYPT14 key from ciphertext.

Implementation: mobile_forensic/whatsapp_crypt.py; integrity.py; parsers/messaging.py; frontend JobIntakePage.tsx

WhatsApp / Derived payloads before artifact parsing

The materialize stage decrypts registered case bytes before native parsing, OCR, media review and text-only RAG. Original encrypted files remain in the case.



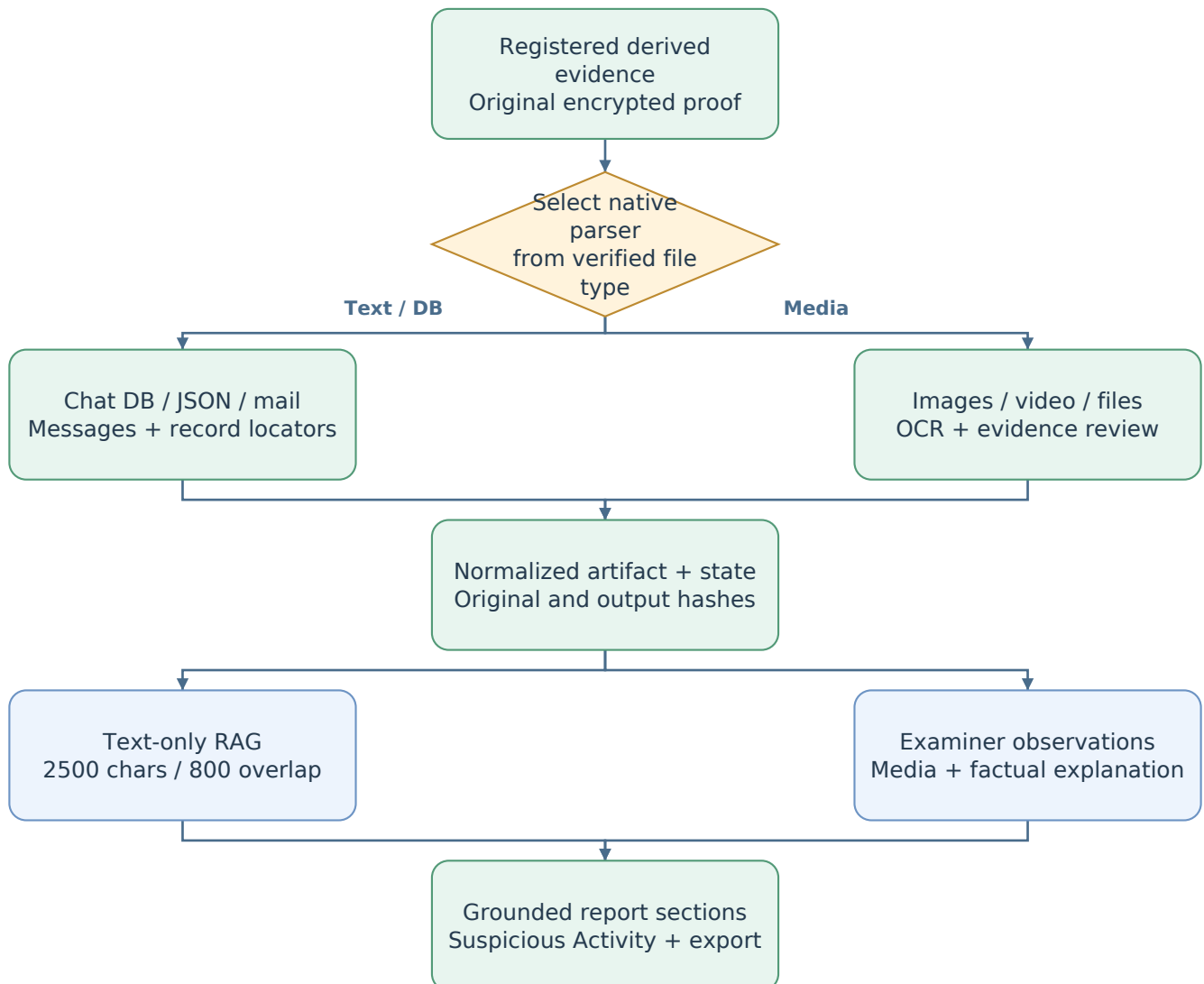
ZIPs Retain the authenticated archive even if a member cannot be expanded. Reject traversal, Windows device names, symlinks, duplicate names and separately encrypted members. Report limits explicitly.

Source boundary After extraction finalization, the server reads sealed sources or verified registered derivations. It never reopens the original phone/dump folder for a newly created working copy.

Implementation: whatsapp_derivation.py; forensic_serial_stages.py; sqlite_counts.py; artifact_parse.py; artifact_preview.py

Recovered data / artifacts, RAG and report evidence

Recovered content follows the existing mobile processing and report paths. A recovered photo or message becomes a suspicious observation only when the evidence review supports that finding.



State Older backup rows are historical. Explicit deletion flags and retained residuals keep their distinct labels. ZIP JSON changesets are captured as structured evidence; unrecognized incremental schemas are not silently replayed into a base database.

Reports Newly recovered evidence requires a new report run. Source-linked images and descriptions use the existing shared Disk/Mobile layout and Suspicious Activity section.

Implementation: `mobile_forensic/parsers/`; `mobile_rag.py`; `suspicious_activity.py`; `report_generator.py`

Run the supplied case / Windows or Python environment

Use the updated application environment, or install its backend requirements. Python 3.11+ is required. The fixture manifest clearly marks the included fixed keys as public synthetic data.

Step	Command / expected result
Synthetic acceptance	<code>python scripts/test-whatsapp-decryption.py --out C:/Aetheris-QA/whatsapp-case</code>
Expected test output	status=passed; 12 checks; whatsapp_synthetic_test_results.json and original/derived synthetic files
Provided CRYPT14 file	<code>python scripts/recover-whatsapp.py --source C:/QA/synthetic_inputs/WhatsApp/Databases/msgstore.db.crypt14 --key C:/QA/synthetic_inputs/data/data/com.whatsapp/files/key --out C:/QA/decoded-crypt14</code>
All fixture formats	<code>python scripts/recover-whatsapp.py --source C:/QA/synthetic_inputs --legacy-account whatsapp-qa@example.test --out C:/QA/decoded-all</code>
Keyless negative case	Repeat the CRYPT14 command without --key. Expect exit code 2, no derived database and key_material_missing_or_invalid.
Application case	Acquire/import the originals; capture or upload matching keys in Intake; save; Reprocess mobile evidence when idle; inspect per-file status, WhatsApp Messages, Files and report observations.

Output directory Keep recovery output outside the input evidence directory. Original backup and key files are never rewritten.

Test integrity The scripts verify expected plaintext hashes, chats, surviving deleted rows, attachment references and unchanged input hashes. A supplied key from another installation is expected to fail.

Implementation: scripts/recover-whatsapp.py; scripts/test-whatsapp-decryption.py

Failure cases / honest results for every source

Input condition	Expected behavior
CRYPT14 only; no matching key	Keep encrypted source; key_material_missing_or_invalid; no fabricated key or plaintext.
ADB error saved as key	Reject during acquisition/Intake. It is not 158-byte usable key evidence, even if padded to that length.
Wrong installation key	Try other distinct collected keys. If none match, keep key_mismatch_or_damaged_backup.
Altered ciphertext / tag / MD5	Do not expose unauthenticated data; keep the original and exact diagnostic.
CRYPT5 without a matching 24-byte key or account	legacy_account_required; a modern 32-byte backup key is insufficient.
Unknown CRYPT version	unsupported_crypt_version; preserve encrypted bytes and report the unsupported capability.
Unsafe / oversize ZIP members	Retain authenticated ZIP, expand only permitted CRC-checked members and list every expansion gap.
Missing or tampered derived object	Reject namespace, source-link, size or hash mismatch before parsing/preview.

Limits WhatsApp backup keys do not decrypt Android filesystem encryption, unrelated applications, password-protected documents or transport-encrypted media with separate keys. Missing/overwritten data cannot be reconstructed by a parser.

Implementation: backend/tests/test_whatsapp_payloads.py; test_whatsapp_derivation.py; test_whatsapp_key_intake.py

Verification scope / performance and live acceptance

Verified in this update	Practical scope
Independent synthetic encoders	Original .crypt and CRYPT5/7/8/9/10/11/12/14/15 use independent cryptography AES/HKDF fixtures; pre-existing independent wa-crypt-tools vectors remain covered.
Portable 12-check case	Key offset, keyless failure, ten supported layouts, ZIP/mail/image/JSON/binary recovery, chats, deleted row, media reference, RAG text and preserved hashes.
PostgreSQL-compatible service checks	Real service SQL, multi-key Intake, deduplicated registration, parser storage, message browser, RAG chunks, case namespace and tamper rejection.
Frontend compilation	Unified, Android and iOS builds include CRYPT5 48-hex/account input, per-file authentication status and preserved key uploads.
Resource changes	Documented IV dispatch; bounded fallback; source/key-set cache; no duplicate DB parses; bounded ZIP expansion; cooperative pause and per-source progress.
Live hardware still required	No physical phone, native Windows phone pull or GPU session was connected. Real case decryption also needs its matching keys; synthetic QA secrets are unusable for real cases.

Coverage This verifies supported code paths. It does not establish recovery of every possible encrypted format or every deleted mobile artifact.

Implementation: WHATSAPP-CRYPT-RELEASE.md; WHATSAPP-CRYPT-MANIFEST.json; LIVE-PHONE-GPU-VERIFICATION.md